

Министерство науки и высшего образования Российской Федерации
Федеральное государственное бюджетное образовательное учреждение
высшего образования

ТОМСКИЙ ГОСУДАРСТВЕННЫЙ УНИВЕРСИТЕТ
СИСТЕМ УПРАВЛЕНИЯ И РАДИОЭЛЕКТРОНИКИ (ТУСУР)

Кафедра автоматизированных систем управления (АСУ)

**ПРИБЛИЖЕНИЕ ФУНКЦИЙ, ЧИСЛЕННОЕ
ДИФФЕРЕНЦИРОВАНИЕ И ИНТЕГРИРОВАНИЕ**

Отчёт по лабораторной работе № 2
по дисциплине «Численные методы»

Студент гр. з-430П8-1

_____ Е.М. Тимошенко

15.09.2026

Руководитель

Канд. техн. наук, доцент кафедры
АСУ

_____ В.В. Романенко

Оценка _____

«____» _____ 2026 г.

Оглавление

ВВЕДЕНИЕ	3
1 ПОСТАНОВКА ЗАДАЧ	4
1.1 Приближение функций	4
1.2 Численное дифференцирование	4
1.3 Численное интегрирование	6
2 ТЕОРЕТИЧЕСКИЕ СВЕДЕНИЯ	7
2.1 Интерполяционный полином Ньютона	7
2.2 Производные интерполяционного полинома	8
2.3 Квадратурные формулы и уточнение шага	8
2.4 Формула Гаусса – Лежандра	9
3 ПРОГРАММНАЯ РЕАЛИЗАЦИЯ	11
3.1 Структура программы	11
3.2 Алгоритм интерполяции и дифференцирования	11
3.3 Алгоритм численного интегрирования	12
4 РЕЗУЛЬТАТЫ РАБОТЫ ПРОГРАММЫ	14
4.1 Результаты интерполяции	14
4.2 Результаты численного дифференцирования	15
4.3 Результаты численного интегрирования	16
4.4 Проверка правильности программы	18
Заключение	19
Список использованных источников	20
Приложение А (обязательное) Листинг вычислительных модулей	21
А.1 Модуль интерполяции и дифференцирования	21
А.2 Модуль численного интегрирования	28

ВВЕДЕНИЕ

Цель работы – изучить методы интерполяции, численного дифференцирования и численного интегрирования, реализовать соответствующие алгоритмы на языке Go и оценить точность результатов. В работе выполняются задания 6.3–6.5 варианта 9 из методических указаний [1].

Для заданий 6.3 и 6.4 применяется интерполяционный полином Ньютона на разделённых разностях. Для задания 6.5 реализованы составные квадратурные формулы средних прямоугольников, трапеций и Симпсона, а также формула Гаусса – Лежандра. Расчётная часть программы написана на языке Go, результаты представлены в локальном веб-интерфейсе. Отчёт оформлен в соответствии с ОС ТУСУР 01-2021 [4].

Вариант задания – 9.

1 ПОСТАНОВКА ЗАДАЧ

1.1 Приближение функций

Задание 6.3. Написать программу интерполяции таблично заданной функции с помощью полиномов Ньютона или Лагранжа [1, с. 102].

Входные данные:

- исходная сетка узлов интерполяции;
- значения интерполируемой функции;
- новая сетка узлов, на которой необходимо вычислить значения функции;
- порядок полинома.

Выходные данные:

- новая сетка;
- значения полинома на новой сетке;
- погрешность интерполирования.

Для варианта 9 задана функция $f(x) = \sqrt{x} + 1$ на отрезке $[1; 2]$. Шаг исходной сетки равен 0,1. Новая сетка определяется выражением $x_j = 1 + jh/2$, $j = 0, \dots, 20$, поэтому её шаг равен 0,05.

1.2 Численное дифференцирование

Задание 6.4. Написать программу вычисления первой и второй производной табличной функции с помощью полинома Ньютона или Лагранжа [1, с. 104].

Входные данные:

- исходная сетка узлов функции;
- значения дифференцируемой функции;
- новая сетка узлов, на которой необходимо вычислить значения производных функции;
- порядок полинома.

Выходные данные:

- новая сетка;
- значения производных полинома на новой сетке;
- погрешность дифференцирования.

Варианты функций и сеток для задания 6.4 совпадают с вариантами задания 6.3 [1, с. 105].

1.3 Численное интегрирование

Задание 6.5 состоит из двух частей [1, с. 105–106]. В первой части требуется написать программу вычисления интеграла по одной из квадратурных формул: трапеций, Симпсона или прямоугольников с автоматическим выбором шага интегрирования.

Входные данные первой части:

- начальное количество интервалов n_0 ;
- сетка узлов либо шаг сетки и границы интервала;
- значения функции либо аналитическая функция;
- относительная точность.

Выходные данные первой части:

- значение интеграла;
- конечное количество интервалов;
- точность интегрирования.

Во второй части требуется написать программу вычисления интеграла по формуле Гаусса. Входными данными являются порядок формулы, границы интервала, подынтегральная функция и относительная точность. Выходными данными являются значение интеграла и точность интегрирования.

Для варианта 9 необходимо вычислить интеграл

$$I = \int_{-0,75}^{0,75} \frac{dx}{\sqrt{1-x^2} \cdot \sqrt{1+x^2}}, \quad n_0 = 8 \quad (1.1)$$

Во второй части используется формула Гаусса порядка 8.

2 ТЕОРЕТИЧЕСКИЕ СВЕДЕНИЯ

2.1 Интерполяционный полином Ньютона

Интерполяция позволяет восстановить приближённые значения функции между известными узлами. Для произвольной сетки полином Ньютона степени m записывается через разделённые разности [2]:

$$P_m(x) = c_0 + c_1(x - x_0) + \dots + c_m \prod_{k=0}^{m-1} (x - x_k) \quad (2.1)$$

где $c_0 = f(x_0)$, $c_1 = f[x_0, x_1]$, ..., $c_m = f[x_0, \dots, x_m]$ – разделённые разности.

Разделённая разность порядка k вычисляется рекуррентно:

$$f[x_i, \dots, x_{i+k}] = \frac{f[x_{i+1}, \dots, x_{i+k}] - f[x_i, \dots, x_{i+k-1}]}{x_{i+k} - x_i} \quad (2.2)$$

В программе для каждой точки новой сетки выбираются $m + 1$ ближайших исходных узлов. Они образуют непрерывное окно исходной таблицы. При равных расстояниях предпочтение отдаётся левому узлу. Такой подход позволяет задавать степень полинома независимо от общего количества исходных точек.

Абсолютная погрешность интерполяции определяется по формуле

$$\Delta(x_j) = |P_m(x_j) - f(x_j)| \quad (2.3)$$

2.2 Производные интерполяционного полинома

Табличная функция не содержит готовых значений производных. Поэтому сначала строится полином Ньютона, а затем вычисляются производные этого полинома. Для вычисления без раскрытия скобок используется вложенная форма полинома. Применение вложенной формы является способом программной реализации, а не дополнительным требованием задания.

$$P_k(t) = c_k + (t - z_k)P_{k+1}(t) \quad (2.4)$$

После дифференцирования выражения (2.4) получаются рекуррентные формулы

$$P'_k(t) = P_{k+1}(t) + (t - z_k)P'_{k+1}(t) \quad (2.5)$$

$$P''_k(t) = 2P'_{k+1}(t) + (t - z_k)P''_{k+1}(t) \quad (2.6)$$

Координата нормируется по формуле $t = (x - x_0)/s$, где s – ширина выбранного окна. По правилу цепочки производные переводятся обратно к координате x :

$$\frac{dP}{dx} = \frac{1}{s} \frac{dP}{dt}; \quad \frac{d^2P}{dx^2} = \frac{1}{s^2} \frac{d^2P}{dt^2} \quad (2.7)$$

Для функции варианта 9 точные производные имеют вид

$$f'(x) = \frac{1}{2\sqrt{x}}; \quad f''(x) = -\frac{1}{4x\sqrt{x}} \quad (2.8)$$

Абсолютные погрешности численного дифференцирования равны

$$\Delta_1 = |P'_m - f'|; \quad \Delta_2 = |P''_m - f''| \quad (2.9)$$

2.3 Квадратурные формулы и уточнение шага

Численное интегрирование заменяет интеграл конечной взвешенной суммой значений функции. В первой части задания выбрана составная

формула Симпсона. Отрезок $[a; b]$ делится на чётное число n интервалов с шагом $h = (b - a)/n$.

$$I_n = \frac{h}{3} [f(x_0) + f(x_n) + 4 \sum_{i=1}^{n/2} f(x_{2i-1}) + 2 \sum_{i=1}^{n/2-1} f(x_{2i})] \quad (2.10)$$

Для автоматического выбора шага сначала вычисляется I_n , затем число интервалов удваивается и вычисляется I_{2n} . Погрешность уточнённого значения оценивается правилом Рунге четвёртого порядка:

$$R = \frac{|I_{2n} - I_n|}{2^4 - 1} = \frac{|I_{2n} - I_n|}{15} \quad (2.11)$$

Уточнение продолжается, пока относительная оценка не удовлетворяет условию

$$\frac{R}{|I_{2n}|} \leq \varepsilon \quad (2.12)$$

2.4 Формула Гаусса – Лежандра

В квадратуре Гаусса узлами являются корни полинома Лежандра, а веса выбираются так, чтобы формула порядка m точно интегрировала полиномы степени до $2m - 1$ [2]. На стандартном отрезке применяется формула

$$\int_{-1}^1 f(t) dt \approx \sum_{j=1}^m w_j f(t_j) \quad (2.13)$$

Для перехода к произвольному отрезку $[a; b]$ используется линейная замена переменной:

$$x = \frac{a + b}{2} + \frac{b - a}{2} t \quad (2.14)$$

В программе сначала применяется восьмиузловая формула ко всему отрезку. Для контроля точности число панелей последовательно удваивается, а оценкой служит модуль разности двух последовательных приближений.

3 ПРОГРАММНАЯ РЕАЛИЗАЦИЯ

3.1 Структура программы

Приложение реализовано на языке Go [3]. Пакет `internal/numerics` содержит численные алгоритмы, пакет `internal/webui` – HTTP-обработчики и встроенные статические ресурсы, а `cmd/lab` – точку запуска. Внешние зависимости Go не используются. Полный листинг основных вычислительных модулей приведён в приложении А.

Таблица 3.1 – Назначение основных исходных файлов

Файл	Назначение
<code>interpolation.go</code>	Полином Ньютона, его производные, выбор локальных узлов и расчёт погрешностей
<code>integration.go</code>	Формулы Симпсона, трапеций, прямоугольников, Гаусса и уточнение шага
<code>expression.go</code>	Разбор пользовательских выражений и автоматическое дифференцирование
<code>server.go</code>	Приём входных данных и передача результатов веб-интерфейсу
<code>numerics_test.go</code>	Автоматические проверки численных алгоритмов

3.2 Алгоритм интерполяции и дифференцирования

Последовательность выполнения заданий 6.3 и 6.4 представлена на рисунке 3.1.

Алгоритм заданий 6.3 и 6.4

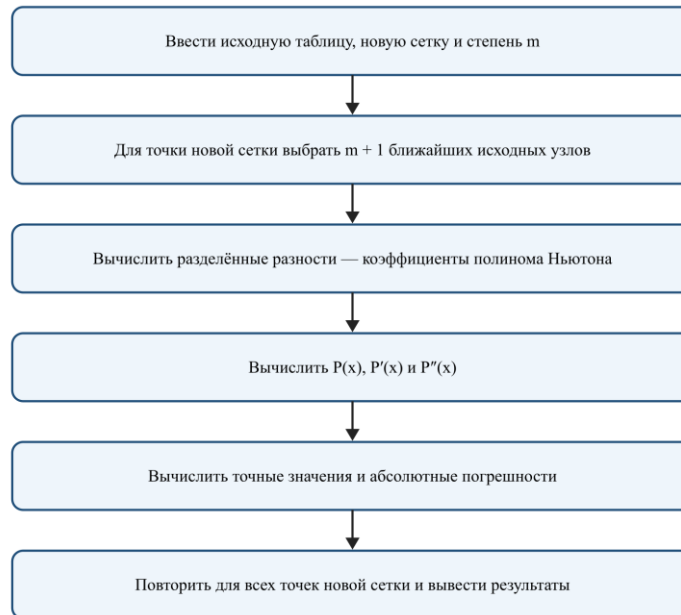


Рисунок 3.1 – Алгоритм интерполяции и численного дифференцирования

Функция `Newton` возвращает сразу три величины: значение полинома, первую и вторую производные. Совместное вычисление исключает повторное построение одного и того же полинома, однако результаты заданий остаются различными: значение относится к 6.3, а две производные – к 6.4.

3.3 Алгоритм численного интегрирования

Последовательность автоматического уточнения шага показана на рисунке 3.2.

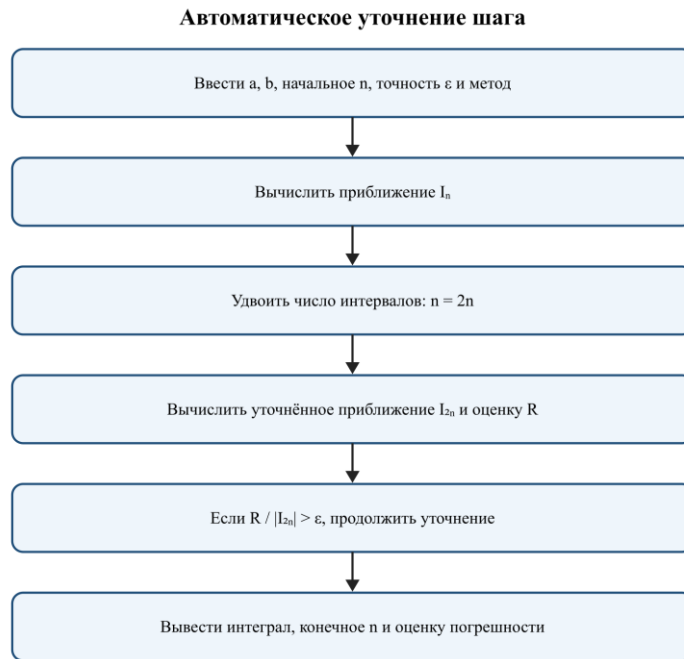


Рисунок 3.2 – Алгоритм автоматического уточнения шага интегрирования

Для формулы Симпсона порядок оценки Рунге равен четырём. Для средних прямоугольников и трапеций он равен двум. Для составной формулы Гаусса используется разность последовательных приближений без деления на $2^{2m} - 1$, чтобы не получить преждевременно заниженную оценку до наступления асимптотического режима.

4 РЕЗУЛЬТАТЫ РАБОТЫ ПРОГРАММЫ

4.1 Результаты интерполяции

Расчёт выполнен при степени локального полинома $m = 3$. Для каждой точки новой сетки использованы четыре ближайших исходных узла. На рисунке 4.1 показаны исходная функция и интерполяционный полином.

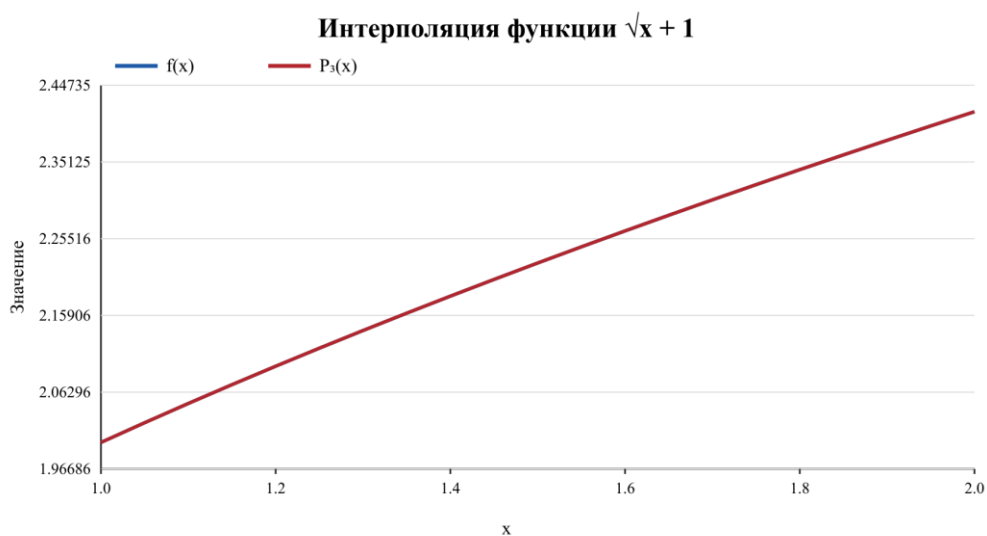


Рисунок 4.1 – Интерполяция функции варианта 9 полиномами Ньютона третьей степени

Таблица 4.1 – Значения интерполяционного полинома на новой сетке

x	$P_3(x)$	$f(x)$	Δ
1	2	2	0
1,05	2,024692660787	2,024695076596	$2,41581 \cdot 10^{-6}$
1,1	2,04880884817	2,04880884817	0
1,25	2,118035003512	2,11803398875	$1,01476 \cdot 10^{-6}$
1,5	2,224744871392	2,224744871392	0
1,75	2,322875966787	2,322875655532	$3,11254 \cdot 10^{-7}$
1,9	2,378404875209	2,378404875209	0
1,95	2,396423593034	2,396424004377	$4,11343 \cdot 10^{-7}$
2	2,414213562373	2,414213562373	0

Максимальная абсолютная погрешность на новой сетке составила $2,41581 \cdot 10^{-6}$. В исходных узлах значение полинома совпадает с табличным значением с точностью вычислений float64.

4.2 Результаты численного дифференцирования

Сравнение приближённой и точной первой производной представлено на рисунке 4.2.

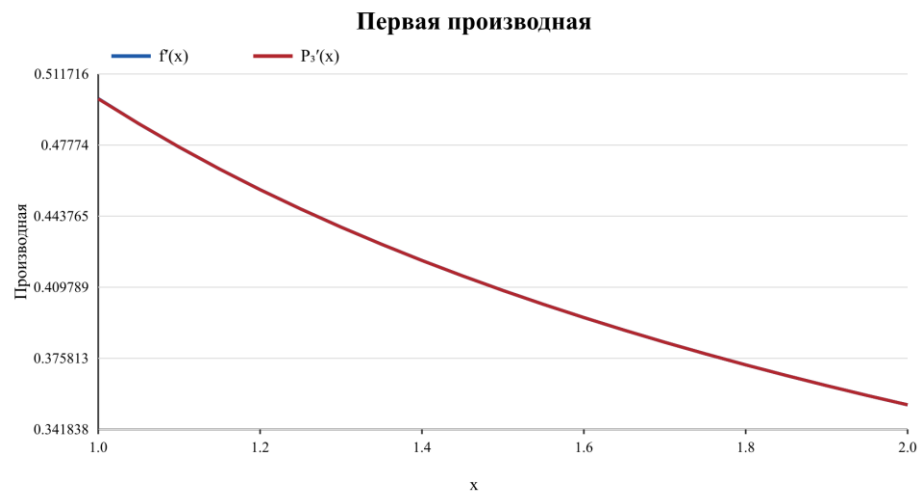


Рисунок 4.2 – Первая производная интерполяционного полинома и исходной функции

Сравнение вторых производных показано на рисунке 4.3.

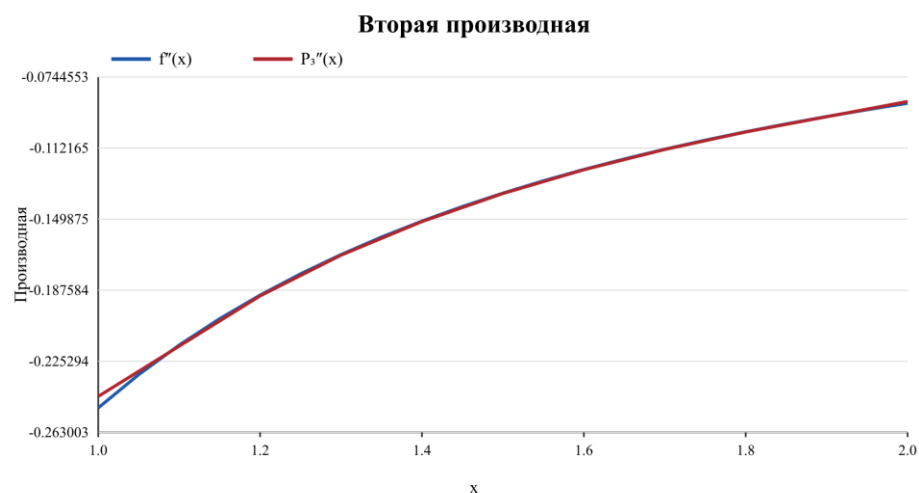


Рисунок 4.3 – Вторая производная интерполяционного полинома и исходной функции

Таблица 4.2 – Результаты численного дифференцирования

x	$P_3'(x)$	$f'(x)$	$P_3''(x)$	$f''(x)$
1	0,499840137	0,5	-0,243920591	-0,25
1,05	0,487977388	0,487950036	-0,230589362	-0,23235716
1,1	0,476781201	0,476731295	-0,217258133	-0,216696043
1,25	0,447213027	0,447213595	-0,179786766	-0,178885438
1,5	0,408227379	0,40824829	-0,13627221	-0,136082763
1,75	0,377964348	0,377964473	-0,108266413	-0,107989849
1,9	0,362729187	0,362738125	-0,095540155	-0,095457401
1,95	0,358053199	0,358057437	-0,091499394	-0,091809599
2	0,353579248	0,353553391	-0,087458634	-0,088388348

Максимальная абсолютная погрешность первой производной равна 0,000159863, второй производной – 0,006079409. Погрешность производных может быть ненулевой даже в исходных узлах, поскольку условие интерполяции обеспечивает совпадение значений функции, но не её наклона и кривизны.

4.3 Результаты численного интегрирования

Для первой части задания использована составная формула Симпсона с начальным количеством интервалов $n_0 = 8$ и относительной точностью $\varepsilon = 10^{-6}$. Ход уточнения приведён в таблице 4.3.

Таблица 4.3 – Уточнение интеграла по формуле Симпсона

n	h	I_n	Оценка R
8	0,1875	1,5560836092211	–
16	0,09375	1,5552573365958	$5,50848 \cdot 10^{-5}$
32	0,046875	1,555181846842	$5,03265 \cdot 10^{-6}$
64	0,0234375	1,5551763583526	$3,65899 \cdot 10^{-7}$

Сходимость последовательных приближений показана на рисунке 4.4.

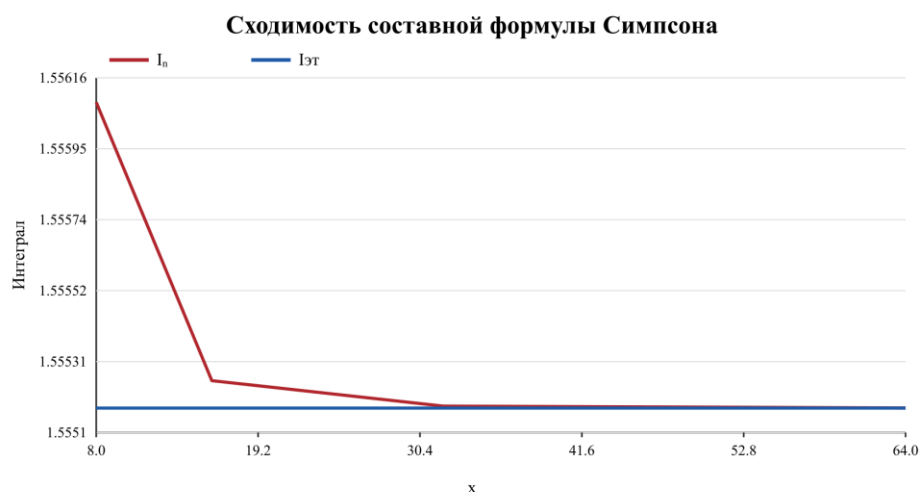


Рисунок 4.4 – Сходимость составной формулы Симпсона

Точность достигнута при $n = 64$. Получено $I = 1,5551763583526$. Независимое эталонное значение равно $I_{\text{эт}} = 1,5551759744797$, абсолютное отклонение составляет $3,83873 \cdot 10^{-7}$.

Во второй части применена формула Гаусса – Лежандра порядка 8. Первый расчёт выполнен на одной панели, второй – на двух панелях. Результаты представлены в таблице 4.4.

Таблица 4.4 – Уточнение интеграла по формуле Гаусса

Панели	h	I_n	$ I_{2n} - I_n $
1	1,5	1,555175053856	—
2	0,75	1,5551759698692	$9,16013 \cdot 10^{-7}$

Получено $I = 1,5551759698692$. Абсолютное отклонение от эталона равно $4,61048 \cdot 10^{-9}$; относительное отклонение равно $2,96460 \cdot 10^{-9}$. Таким образом, заданная относительная точность 10^{-6} достигнута.

4.4 Проверка правильности программы

Интерполяция и дифференцирование проверены на полиноме $f(x) = 2x^3 - 3x^2 + 4x - 7$. Полином Ньютона третьей степени должен точно восстановить эту функцию и производные

$$f'(x) = 6x^2 - 6x + 4; \quad f''(x) = 12x - 6 \quad (4.1)$$

Автоматический тест сравнивает значения в шести точках с допуском 10^{-12} . Формула Гаусса порядка m проверяется на одночленах всех степеней от 0 до $2m - 1$. Также проверяются формулы Симпсона, трапеций и прямоугольников, ограничения входных данных и HTTP-интерфейс. Команды `go test ./...` и `go vet ./...` выполняются без ошибок.

Заключение

В ходе лабораторной работы реализованы задания 6.3, 6.4 и 6.5 варианта 9. Табличная функция интерполирована локальными полиномами Ньютона третьей степени на новой сетке. Наибольшая абсолютная погрешность интерполяции составила $2,41581 \cdot 10^{-6}$. Первая и вторая производные получены дифференцированием интерполяционного полинома; максимальные погрешности составили соответственно 0,000159863 и 0,006079409.

Интеграл функции варианта вычислен составной формулой Симпсона и формулой Гаусса – Лежандра восьмого порядка. При точности 10^{-6} формула Симпсона потребовала 64 интервала, а формула Гаусса – две панели. Сравнение с независимым эталоном и автоматические тесты подтвердили правильность реализации. Таким образом, поставленные задачи выполнены полностью.

Список использованных источников

1. Вычислительная математика: методические указания к контрольной и лабораторным работам. Томск: ТУСУР, 2019. 117 с.
2. Вычислительные методы: учебное пособие. Томск: ТУСУР. 198 с.
3. The Go Programming Language Documentation [Электронный ресурс]. URL: <https://go.dev/doc/> (дата обращения: 15.09.2026).
4. ОС ТУСУР 01-2021. Работы студенческие по направлениям подготовки и специальностям технического профиля. Общие требования и правила оформления. Томск: ТУСУР, 2021. 52 с.

Приложение А (обязательное)

Листинг вычислительных модулей

В приложении приведены модули, реализующие вычисления заданий 6.3–6.5.

А.1 Модуль интерполяции и дифференцирования

// Package numerics implements the methods from tasks 6.3–6.5, variant 9.

```
package numerics
```

```
import (  
    "fmt"  
    "math"  
    "sort"  
)
```

```
type Point struct {  
    X float64 `json:"x"`  
    Y float64 `json:"y"`  
}
```

```
type InterpolationInput struct {  
    Nodes    []Point `json:"nodes"`  
    Grid     []float64 `json:"grid"`  
    Degree    int     `json:"degree"`  
    Reference bool    `json:"reference"`  
    Expression string  `json:"expression"`  
    UseFunction bool   `json:"useFunction"`  
}
```

```
type InterpolationRow struct {  
    X float64 `json:"x"`
```

```

    Value    float64 `json:"value"`
    First    float64 `json:"first"`
    Second   float64 `json:"second"`
    Exact    *float64 `json:"exact"`
    ExactFirst *float64 `json:"exactFirst"`
    ExactSecond *float64 `json:"exactSecond"`
    Error     *float64 `json:"error"`
    ErrorFirst *float64 `json:"errorFirst"`
    ErrorSecond *float64 `json:"errorSecond"`
    Start     int     `json:"start"`
    End       int     `json:"end"`
}

type InterpolationResult struct {
    Rows    []InterpolationRow `json:"rows"`
    Curve    []InterpolationRow `json:"curve"`
    Nodes    []Point             `json:"nodes"`
    Degree   int                 `json:"degree"`
    Reference bool                 `json:"reference"`
    MaxError [3]float64         `json:"maxError"`
    Expression string         `json:"expression"`
}

func finite(v float64) bool { return !math.IsNaN(v) && !math.IsInf(v, 0) }
func Function(x float64) float64 { return math.Sqrt(x) + 1 }
func Derivatives(x float64) (float64, float64) {
    return 1 / (2 * math.Sqrt(x)), -1 / (4 * x * math.Sqrt(x))
}

func DefaultInterpolation() InterpolationInput {

```

```

    in := InterpolationInput{Degree: 3, Reference: true, Expression:
DefaultFunction, UseFunction: true}
    for i := 0; i <= 10; i++ {
        x := 1 + float64(i)/10
        in.Nodes = append(in.Nodes, Point{x, Function(x)})
    }
    for i := 0; i <= 20; i++ {
        in.Grid = append(in.Grid, 1+float64(i)/20)
    }
    return in
}

// Newton evaluates a polynomial and its analytic first and second derivatives.
// Affine scaling keeps divided differences away from very small powers of h.
func Newton(nodes []Point, x float64) (value, first, second float64) {
    n := len(nodes)
    origin := nodes[0].X
    scale := nodes[n-1].X - origin
    if n == 1 {
        return nodes[0].Y, 0, 0
    }
    z := make([]float64, n)
    c := make([]float64, n)
    for i, p := range nodes {
        z[i] = (p.X - origin) / scale
        c[i] = p.Y
    }
    for k := 1; k < n; k++ {
        for i := n - 1; i >= k; i-- {
            c[i] = (c[i] - c[i-1]) / (z[i] - z[i-k])
        }
    }
}

```

```

        }
    }
    t := (x - origin) / scale
    value = c[n-1]
    for i := n - 2; i >= 0; i-- {
        second = second*(t-z[i]) + 2*first
        first = first*(t-z[i]) + value
        value = value*(t-z[i]) + c[i]
    }
    return value, first / scale, second / (scale * scale)
}

// Select degree+1 nearest nodes, retaining their natural order. In one
// dimension these form a contiguous window. Ties choose the left node.
func window(nodes []Point, x float64, degree int) int {
    r := sort.Search(len(nodes), func(i int) bool { return nodes[i].X >= x })
    l := r - 1
    for i := 0; i <= degree; i++ {
        if l < 0 {
            r++
        } else if r >= len(nodes) || x-nodes[l].X <= nodes[r].X-x {
            l--
        } else {
            r++
        }
    }
    return l + 1
}

```

```

func Interpolate(in InterpolationInput) (InterpolationResult, error) {

```

```

if in.Expression == "" {
    in.Expression = DefaultFunction
}
out := InterpolationResult{Degree: in.Degree, Reference: in.Reference,
Expression: in.Expression}
var formula *expression
if in.Reference || in.UseFunction {
    var err error
    formula, err = compileExpression(in.Expression)
    if err != nil {
        return out, err
    }
}
in.Nodes = append([]Point(nil), in.Nodes...)
if len(in.Nodes) < 2 || len(in.Nodes) > 201 {
    return out, fmt.Errorf("нужно от 2 до 201 исходного узла")
}
if in.Degree < 1 || in.Degree > 20 || in.Degree >= len(in.Nodes) {
    return out, fmt.Errorf("степень должна быть от 1 до min(20, число
узлов - 1)")
}
if len(in.Grid) < 1 || len(in.Grid) > 2001 {
    return out, fmt.Errorf("новая сетка должна содержать от 1 до 2001
узла")
}
for i, p := range in.Nodes {
    if in.UseFunction {
        p.Y = formula.eval(p.X).v
        in.Nodes[i].Y = p.Y
    }
}

```

```

    if !finite(p.X) || !finite(p.Y) || math.Abs(p.X) > 1e6 || math.Abs(p.Y) >
1e12 {
        return out, fmt.Errorf("некорректные данные узла %d", i)
    }
    if i > 0 && p.X-in.Nodes[i-1].X < 1e-9 {
        return out, fmt.Errorf("исходные x должны строго
возрастать; минимальное расстояние 10-9")
    }
    if in.Reference {
        f := formula.eval(p.X).v
        if !finite(f) || math.Abs(p.Y-f) > 1e-12*math.Max(1,
math.Abs(f)) {
            return out, fmt.Errorf("значения таблицы не совпадают
с формулой; включите вычисление y по формуле или отключите сравнение")
        }
    }
    a, b := in.Nodes[0].X, in.Nodes[len(in.Nodes)-1].X
    for _, x := range in.Grid {
        if !finite(x) || x < a || x > b {
            return out, fmt.Errorf("новая сетка должна лежать внутри
[%g; %g]", a, b)
        }
    }
    eval := func(x float64) (InterpolationRow, error) {
        s := window(in.Nodes, x, in.Degree)
        v, d1, d2 := Newton(in.Nodes[s:s+in.Degree+1], x)
        row := InterpolationRow{X: x, Value: v, First: d1, Second: d2, Start:
s, End: s + in.Degree}
        if !finite(v) || !finite(d1) || !finite(d2) {

```

```

        return row, fmt.Errorf("переполнение: уменьшите степень
или измените узлы")
    }
    if in.Reference {
        analytic := formula.eval(x)
        f, f1, f2 := analytic.v, analytic.d, analytic.dd
        if !finite(f) || !finite(f1) || !finite(f2) {
            return row, fmt.Errorf("функция или её производные
не определены при x = %g; измените интервал или отключите аналитическое
сравнение", x)
        }
        e, e1, e2 := math.Abs(v-f), math.Abs(d1-f1), math.Abs(d2-f2)
        row.Exact = &f
        row.ExactFirst = &f1
        row.ExactSecond = &f2
        row.Error = &e
        row.ErrorFirst = &e1
        row.ErrorSecond = &e2
    }
    return row, nil
}
for _, x := range in.Grid {
    r, e := eval(x)
    if e != nil {
        return out, e
    }
    out.Rows = append(out.Rows, r)
    if in.Reference {
        out.MaxError[0] = math.Max(out.MaxError[0], *r.Error)
        out.MaxError[1] = math.Max(out.MaxError[1], *r.ErrorFirst)
    }
}

```

```

        out.MaxError[2] = math.Max(out.MaxError[2],
*r.ErrorSecond)
    }
}
for i := 0; i <= 400; i++ {
    r, e := eval(a + (b-a)*float64(i)/400)
    if e != nil {
        return out, e
    }
    out.Curve = append(out.Curve, r)
}
out.Nodes = append([]Point(nil), in.Nodes...)
return out, nil
}

```

A.2 Модуль численного интегрирования

```
package numerics
```

```

import (
    "fmt"
    "math"
    "strings"
)

```

```

type IntegrationInput struct {
    A      float64 `json:"a"`
    B      float64 `json:"b"`
    N      int     `json:"n"`
    Tolerance float64 `json:"tolerance"`
}

```

```

    Method    string `json:"method"`
    Order     int    `json:"order"`
    Expression string `json:"expression"`
}

type Iteration struct {
    N      int    `json:"n"`
    H      float64 `json:"h"`
    Value  float64 `json:"value"`
    Estimate *float64 `json:"estimate"`
    Relative *float64 `json:"relative"`
}

type IntegrationResult struct {
    Input          IntegrationInput `json:"input"`
    Value          float64         `json:"value"`
    Reference      float64         `json:"reference"`
    AbsoluteError  float64         `json:"absoluteError"`
    RelativeError  float64         `json:"relativeError"`
    Estimate       float64         `json:"estimate"`
    N              int             `json:"n"`
    Converged      bool            `json:"converged"`
    Message        string          `json:"message"`
    Iterations     []Iteration     `json:"iterations"`
    GaussNodes     []Point         `json:"gaussNodes"`
    Curve          []Point         `json:"curve"`
    HasReference    bool            `json:"hasReference"`
    AbsoluteCriterion bool           `json:"absoluteCriterion"`
}

func Integrand(x float64) float64 { return 1 / math.Sqrt((1-x*x)*(1+x*x)) }

```

```

// ReferencePrimitive uses the independently convergent binomial series
//  $(1-x^4)^{-1/2} = \sum C(2k,k)/4^k x^{4k}$ . It is not a quadrature result.
func ReferencePrimitive(x float64) float64 {
    power := x
    coefficient := 1.0
    sum, correction := 0.0, 0.0
    x4 := x * x * x * x
    for k := 0; k < 100000; k++ {
        term := coefficient * power / float64(4*k+1)
        y := term - correction
        t := sum + y
        correction = (t - sum) - y
        sum = t
        if math.Abs(term)/(1-x4) < 2e-16*math.Max(1, math.Abs(sum)) {
            break
        }
        coefficient *= float64(2*k+1) / float64(2*k+2)
        power *= x4
    }
    return sum
}

```

```

// GaussRule returns Legendre roots and weights on [-1,1].
func GaussRule(order int) []Point {
    rule := make([]Point, order)
    for i := 0; i < (order+1)/2; i++ {
        z := math.Cos(math.Pi * (float64(i) + 0.75) / (float64(order) + 0.5))
        derivative := func(x float64) (float64, float64) {
            p0, p1 := 1.0, x
            for k := 2; k <= order; k++ {

```

```

        p0, p1 = p1, (float64(2*k-1)*x*p1-float64(k-
1)*p0)/float64(k)
    }
    return p1, float64(order) * (x*p1 - p0) / (x*x - 1)
}
for k := 0; k < 100; k++ {
    p, d := derivative(z)
    next := z - p/d
    if math.Abs(next-z) < 2e-15 {
        z = next
        break
    }
    z = next
}
_, d := derivative(z)
w := 2 / ((1 - z*z) * d * d)
rule[i] = Point{-z, w}
rule[order-1-i] = Point{z, w}
}
return rule
}

```

```

func quadrature(f func(float64) float64, a, b float64, n int, method string, rule
[]Point) float64 {
    h := (b - a) / float64(n)
    sum, correction := 0.0, 0.0
    add := func(v float64) { y := v - correction; t := sum + y; correction = (t -
sum) - y; sum = t }
    switch method {
    case "midpoint":

```

```

    for i := 0; i < n; i++ {
        add(f(a + (float64(i)+0.5)*h))
    }
    return h * sum
case "trapezoid":
    add(f(a) / 2)
    add(f(b) / 2)
    for i := 1; i < n; i++ {
        add(f(a + float64(i)*h))
    }
    return h * sum
case "simpson":
    add(f(a))
    add(f(b))
    for i := 1; i < n; i++ {
        w := 2.0
        if i%2 == 1 {
            w = 4
        }
        add(w * f(a+float64(i)*h))
    }
    return h * sum / 3
case "gauss":
    for i := 0; i < n; i++ {
        mid := a + (float64(i)+0.5)*h
        for _, p := range rule {
            add(p.Y * f(mid+h*p.X/2))
        }
    }
    return h * sum / 2

```

```

    }
    return math.NaN()
}

func Integrate(in IntegrationInput) (IntegrationResult, error) {
    if strings.TrimSpace(in.Expression) == "" {
        in.Expression = DefaultIntegrand
    }
    out := IntegrationResult{Input: in, GaussNodes: []Point{}}
    out.HasReference = strings.Join(strings.Fields(in.Expression), "") ==
DefaultIntegrand
    if !finite(in.A) || !finite(in.B) || in.B-in.A < 1e-9 || math.Abs(in.A) > 1e6 ||
math.Abs(in.B) > 1e6 {
        return out, fmt.Errorf("нужны конечные границы  $-1\,000\,000 \leq a < b \leq 1\,000\,000$ ; длина интервала не меньше  $10^{-9}$ ")
    }
    if out.HasReference && (in.A < -.99 || in.B > .99) {
        return out, fmt.Errorf("для функции варианта 9 границы должны удовлетворять  $-0,99 \leq a < b \leq 0,99$ ")
    }
    formula, err := compileExpression(in.Expression)
    if err != nil {
        return out, err
    }
    var evaluationError error
    f := func(x float64) float64 {
        if evaluationError != nil {
            return math.NaN()
        }
        v := formula.eval(x).v

```

```

    if !finite(v) {
        evaluationError = fmt.Errorf("функция не определена или
переполняется при x = %g; проверьте формулу и границы", x)
    }
    return v
}

// Include boundaries even for methods with interior-only nodes.
for i := 0; i <= 400; i++ {
    x := in.A + (in.B-in.A)*float64(i)/400
    v := f(x)
    if evaluationError != nil {
        return out, evaluationError
    }
    out.Curve = append(out.Curve, Point{x, v})
}

if !finite(in.Tolerance) || in.Tolerance < 1e-12 || in.Tolerance > .1 {
    return out, fmt.Errorf("относительная точность должна быть от
10-12 до 0,1")
}

if in.N < 1 || in.N > 4096 {
    return out, fmt.Errorf("начальное число интервалов должно быть
от 1 до 4096")
}

if in.Order < 2 || in.Order > 16 {
    return out, fmt.Errorf("порядок Гаусса должен быть от 2 до 16")
}

p := 2
var rule []Point
switch in.Method {
case "midpoint", "trapezoid":

```

```

case "simpson":
    if in.N%2 != 0 {
        return out, fmt.Errorf("для Симпсона начальное число
интервалов должно быть чётным")
    }
    p = 4
case "gauss":
    rule = GaussRule(in.Order)
    out.GaussNodes = rule
default:
    return out, fmt.Errorf("неизвестный метод интегрирования")
}
if out.HasReference {
    out.Reference = ReferencePrimitive(in.B) - ReferencePrimitive(in.A)
}
n := in.N
if in.Method == "gauss" {
    n = 1
}
previous := quadrature(f, in.A, in.B, n, in.Method, rule)
if evaluationError != nil {
    return out, evaluationError
}
if !finite(previous) {
    return out, fmt.Errorf("переполнение при интегрировании")
}
out.Iterations = append(out.Iterations, Iteration{N: n, H: (in.B - in.A) /
float64(n), Value: previous})
maxIntervals := 1 << 20
if !out.HasReference {

```

```

    maxIntervals = 1 << 16
}
for n <= maxIntervals/2 {
    n *= 2
    value := quadrature(f, in.A, in.B, n, in.Method, rule)
    if evaluationError != nil {
        return out, evaluationError
    }
    if !finite(value) {
        return out, fmt.Errorf("переполнение при интегрировании")
    }
    estimate := math.Abs(value-previous) / float64((int64(1)<<p)-1)
    // For Gauss use the undivided difference: the asymptotic 2^(2m)-1
    // divisor is overly optimistic before reaching the asymptotic regime.
    if in.Method == "gauss" {
        estimate = math.Abs(value - previous)
    }
    out.AbsoluteCriterion = math.Abs(value) < 1e-12
    scale := math.Abs(value)
    if out.AbsoluteCriterion {
        scale = 1
    }
    relative := estimate / scale
    out.Iterations = append(out.Iterations, Iteration{n, (in.B - in.A) /
float64(n), value, &estimate, &relative})
    out.Value = value
    out.N = n
    out.Estimate = estimate
    if out.HasReference {
        out.AbsoluteError = math.Abs(value - out.Reference)
    }
}

```

```

        out.RelativeError = out.AbsoluteError /
math.Abs(out.Reference)
    }
    // The known function permits an independent verification of the
target.
    if relative <= in.Tolerance && (!out.HasReference || out.RelativeError
<= in.Tolerance) {
        out.Converged = true
        break
    }
    previous = value
}
out.Message = "Точность достигнута и проверена по независимому
степенному ряду."
if !out.HasReference {
    out.Message = "Критерий точности выполнен по оценке
последовательных приближений; точный интеграл неизвестен."
}
if out.AbsoluteCriterion {
    out.Message = "Интеграл близок к нулю: ε используется как
абсолютная точность. Критерий выполнен по оценке метода."
}
if !out.Converged {
    out.Message = fmt.Sprintf("Достигнут предел %d интервалов;
заданная точность не достигнута.", maxIntervals)
}
return out, nil
}

```